गौतम बुद्ध विश्वविद्यालय
GAUTAM BUDDHA UNIVERSITY
An Ultimate Destination for Higher Learning

CLASSIFICATION MODEL FOR DETECTION OF DISEASE IN FABACEAE PLANTS USING DEEP CONVOLUTIONAL NEURAL NETWORK AND CONNECTION PRUNING

Dissertation Part-2

(End-Sem)

Presented by

Prakhar Rastogi

[16/ICS/038]

Supervised by

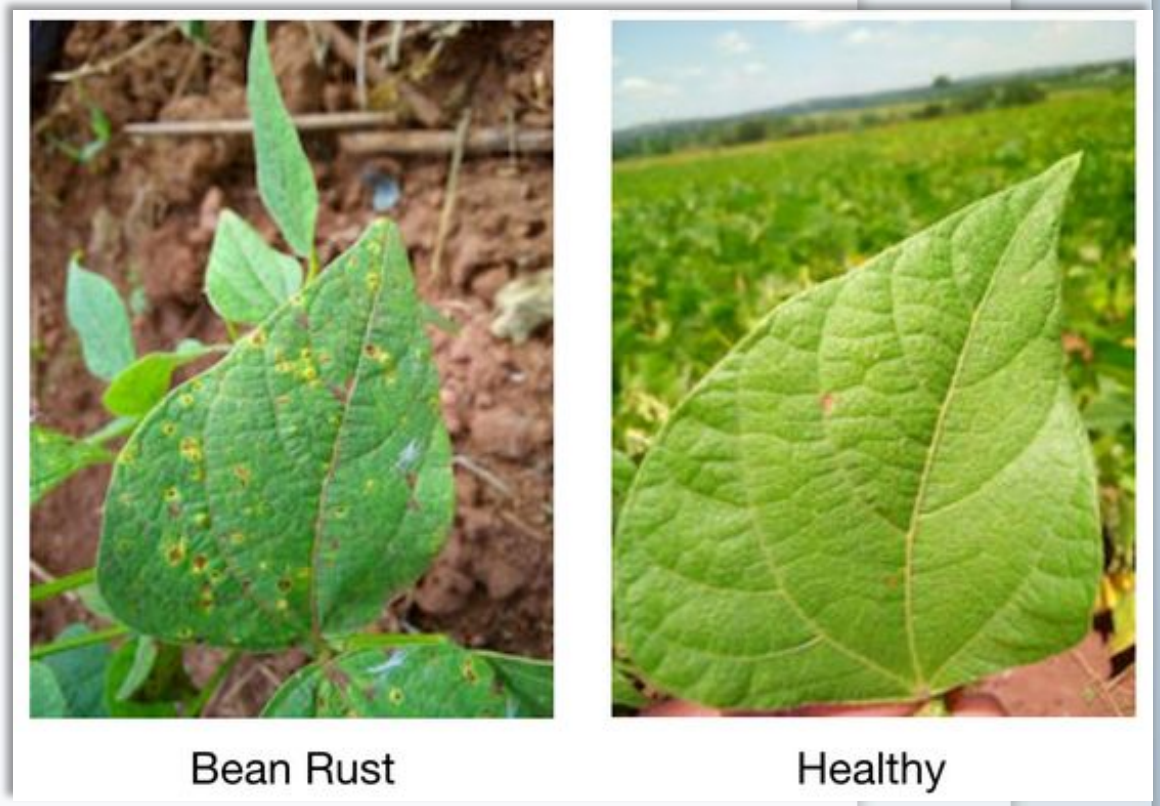
Dr. Arun Solanki

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY
GAUTAM BUDDHA UNIVERSITY
GREATER NOIDA, UTTAR PRADESH, INDIA – 201312
JUNE 2021

CONTENTS

- Introduction
- Deep Learning and Convolutional Neural Networks
- Literature Review
- Research Objectives
- Research Methodology
- Implementation
- Experimental Results
- Conclusion
- List of Publications
- References

Introduction



Introduction

- Beans are an important cereal food crop grown by many small-holder farmers, these are significant source of proteins and Carbohydrates.
- Angular leaf spot (ALS) and bean rust, caused by the fungus and considered serious disease of beans.
- Manual detection of diseases in plants is a difficult and time consuming task.
- Use of machine learning techniques for detection of disease in plants.
- The dataset is of leaf images representing 3 classes: the healthy class of images, and two disease classes including Angular Leaf Spot and Bean Rust diseases.

- The proposed model should be able to distinguish between these 3 classes with high accuracy.

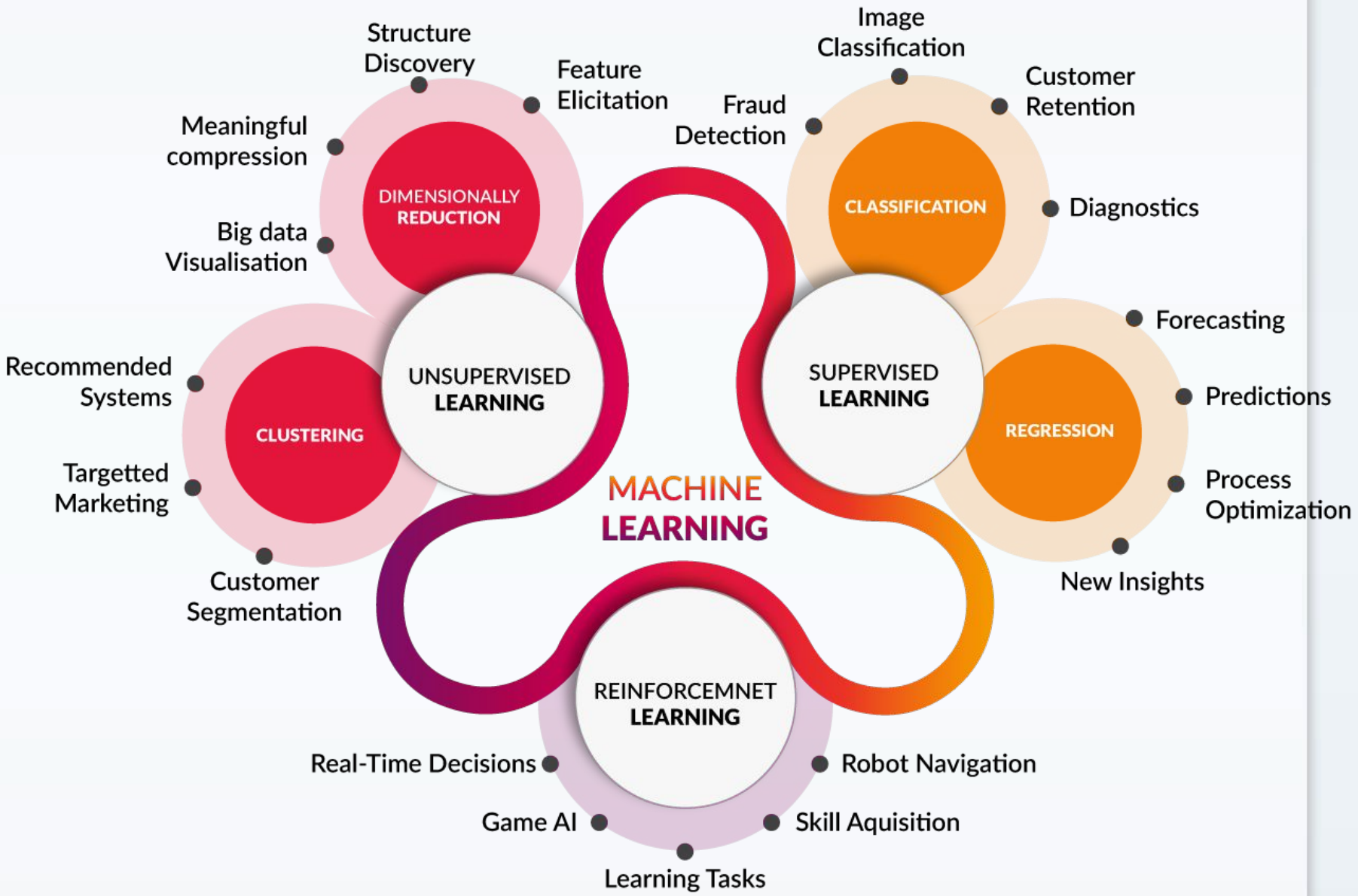
  - Angular Leaf Spot
  - Bean Rust
  - Healthy Leaf

Angular Leaf Spot

Bean Rust

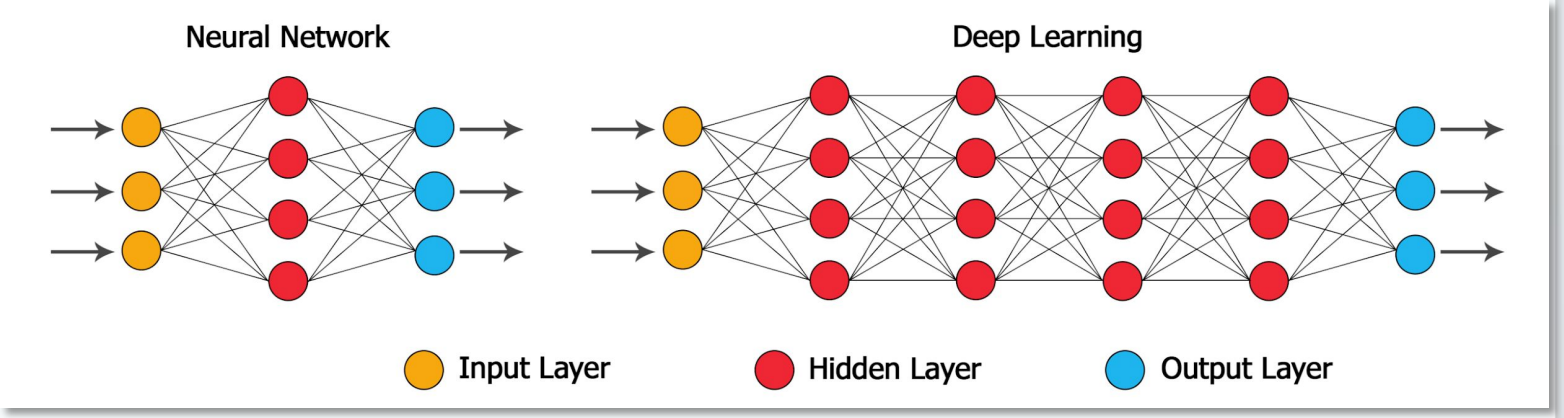
Healthy

Deep Learning
And
Convolutional Neural Network


MACHINE LEARNING
UNSUPERVISED LEARNING
DIMENSIONALLY REDUCTION
Structure Discovery
Feature Elicitation
Meaningful compression
Big data Visualisation
CLUSTERING
Recommended Systems
Targetted Marketing
Customer Segmentation
SUPERVISED LEARNING
CLASSIFICATION
Image Classification
Customer Retention
Fraud Detection
Diagnostics
REGRESSION
Forecasting
Predictions
Process Optimization
New Insights
REINFORCEMNET LEARNING
Real-Time Decisions
Robot Navigation
Game AI
Skill Aquisition
Learning Tasks

Deep Learning

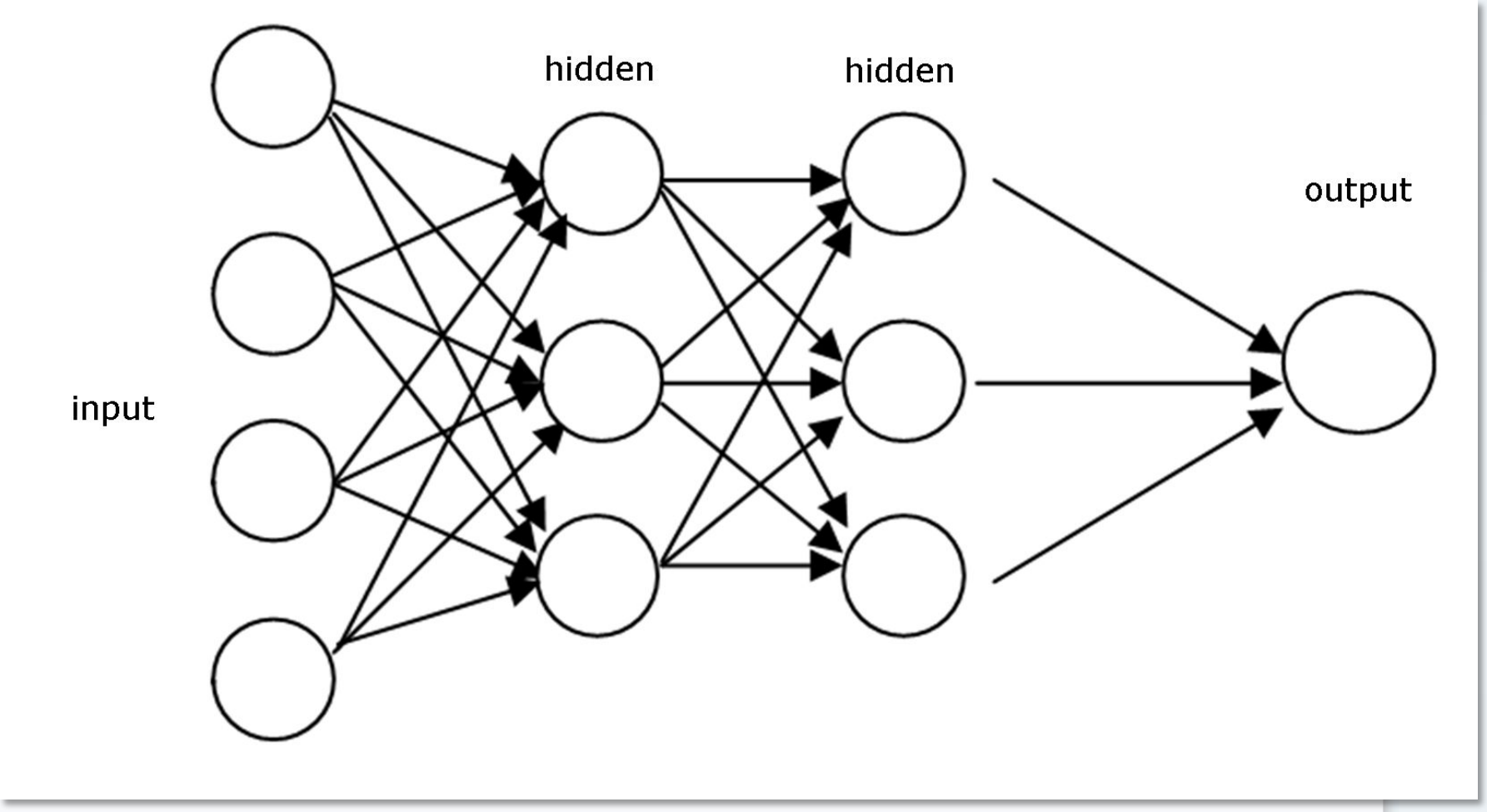
- Deep learning is a subset of machine learning in artificial intelligence that has neural networks, capable of learning from data that is unstructured or unlabeled also known as deep neural learning or deep neural networks.



Feed forward Neural Network

- This type of neural network contains multiple neurons arranged in layers and connections between the nodes and do not form a cycle or loop.

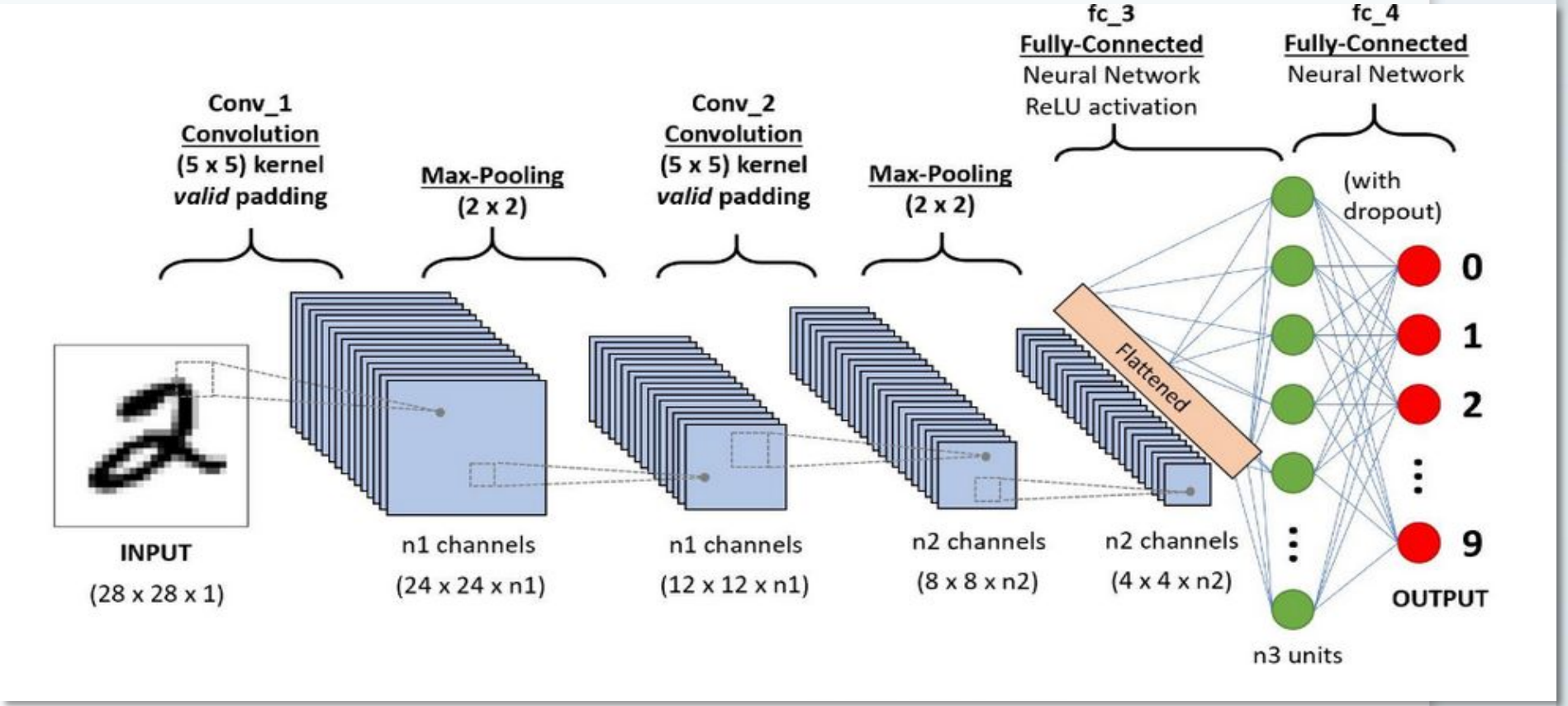
  - Input layer
  - Hidden layer
  - Output layer



Feed forward Neural Network with two hidden layers

Convolutional Neural Networks

- Convolutional neural networks (CNNs) are special kind of multi-layer neural networks, designed to recognize visual patterns directly from pixel images with minimal pre-processing.
- The LeNet architecture is a first and basic architecture for convolutional neural networks.
- LeNet is small and easy to understand, large enough to provide interesting results.
- LeNet is a five-layer network consists of 2 convolutional layers and 3 fully connected layers.
- Convolutional layers, pooling layers and fully connected dense layers are used.



Basic Convolutional Neural Network

Literature Review

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
1	Priyadharshini RA, Arivazhagan S, Arun M, Mirnalini A. Maize leaf disease classification using deep convolutional neural networks. (2019)	Neural Computing and Applications. 2019 Dec;31(12):8887-95. Springer Nature 2019	A deep convolutional neural network (CNN)-based architecture (LeNet) for the classification of maize leaf disease is used . It discussed the potential efficiency of the LeNet architecture for plant leaf disease classification by varying the parameters like kernel size and depth.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
2	Uğuz, Sinan, and Nese Uysal. Classification of olive leaf diseases using deep convolutional neural networks. (2020)	Neural Computing and Applications (2020): 1-17. Springer Nature 2020	Classification of different models were compared by training dataset on three different models (CNN model, Transfer learning-VGG16, Transfer learning-VGG19). Experiments were carried out in two main sections depending on whether dataset was ‘‘with data augmentation’’ or ‘‘without data augmentation’’. VGG16 and VGG19 models with transfer learning produces better results compared to CNN model.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
3	Sardogan, Melike, Adem Tuncer, and Yunus Ozen. Plant leaf disease detection and classification based on CNN with LVQ algorithm. (2018)	3rd International Conference on Computer Science and Engineering (UBMK), pp. 382-385. IEEE, 2018.	In this paper, a tomato leaf diseases detection and classification method is presented based on Convolutional Neural Network(CNN) with Learning Vector Quantization algorithm for classification. It consists of three layers with input, Kohonen (competition) and output layer.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
4	Taylor, Luke, and Geoff Nitschke. "Improving deep learning using generic data augmentation." (2018)	2018 IEEE Symposium Series on Computational Intelligence (SSCI)	An effective method of increasing CNN classification task performance is to make use of Data Augmentation (DA) like flipping , rotating , jittering and cropping the image.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
5	Dongmei Hana , Qigang Liu , Weiguo Fan A New Image Classification Method Using CNN transfer learning and Web Data Augmentation (2017)	Expert Systems With Applications (2017), doi: 10.1016/j.eswa.2017.11.028	In this paper, a novel two-phase classification method is proposed, which takes advantages of transfer learning and data augmentation technique. By transfer learning, the requirement for a large number of training data is fullfilled and a powerful classifier is constructed with data augmentation.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
6	Zhu, Ling, Zhenbo Li, Chen Li, Jing Wu, and Jun Yue. "High performance vegetable classification from images based on alexnet deep learning model." (2018)	International Journal of Agricultural and Biological Engineering 11, no. 4 (2018): 217-223.	This paper proposes an image classification method based on AlexNet convolutional neural network. AlexNet network has 8 layers, the first five layers are the convolutional layers, the latter three layers are the full connected layers.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
7	Sharma, Neha, Vibhor Jain, and Anju Mishra. "An analysis of convolutional neural networks for image classification." (2018)	Procedia computer science 132 (2018): 377-384.	The results suggested that trained networks with transfer learning performed better than existing ones and showed higher rates of accuracy. It can be concluded that the performance of 27 layered networks was not much appreciated. Hence, more the number of layers, more will be the training time and therefore, higher the rate of accuracy in prediction can be achieved.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
8	Krishna, Sajja Tulasi, and Hemantha Kumar Kalluri. "Deep learning and transfer learning approaches for image classification." (2019)	International Journal of Recent Technology and Engineering (IJRTE) 7, no. 5S4 (2019): 427-432.	A brief survey with Deep Neural Network (DNN) in Deep Learning area and the Convolutional Neural Network (CNN) and its architectures, such as LeNet, AlexNet, GoogleNet, VGG16, VGG19, Resnet50 etc. It also includes transfer learning by using the CNN's pre-trained architectures. This survey concludes that the size of data may affect the accuracy and the number of epochs for Image classification task.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
9	Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." (2012)	Advances in neural information processing systems 25 (2012): 1097-1105.	A very deep neural network architecture is developed which contains eight learned layers, five convolutional and three fully-connected layers (AlexNet). This model performs better than LeNet model on ImageNet dataset.

Reference No.	Title , Author and Year	Conference/Journal	Methodology, Findings and Gaps
10	Zhu, Michael, and Suyog Gupta. "To prune, or not to prune: exploring the efficacy of pruning for model compression." .	arXiv preprint arXiv:1710.01878 (2017)	This paper demonstrates that large-sparse models outperform comparably-sized small-dense models across a diverse set of neural network architectures for eg. MobileNet , Inception Net etc.

Research Objectives

Research Objectives

- The research's objective is to propose a solution for classification of disease in plants, which can be formalised as-

● To implement a plant disease detection system for detection of fungal infections in bean leaves.

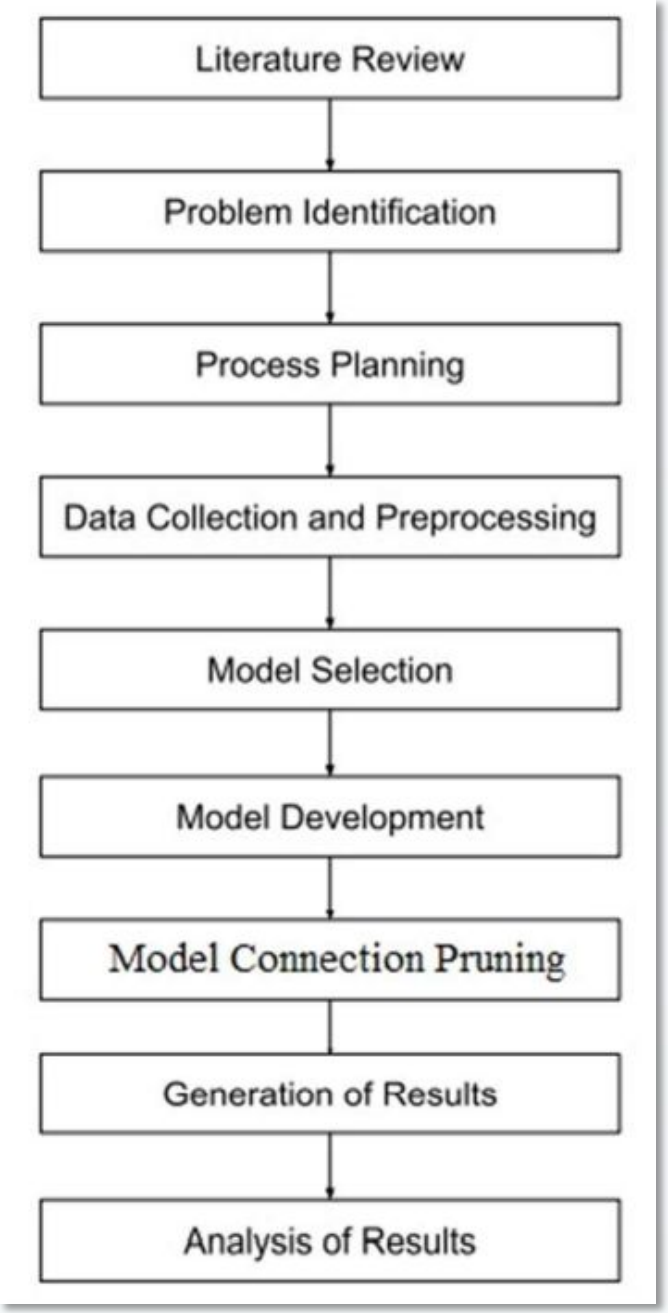
● To enhance the performance of the system by using connection pruning to optimize model training and testing with different feature extractors.

● To validate the proposed model's performance against existing work in the field of plant disease detection.

Time Plan

Months Tasks	Aug 20-Sep 20	Oct 20-Dec 20	Dec 20 – Jan 21	Jan 21 – Feb 21	Feb 20 - Mar 21	Mar 21 – Apr 21	Apr 21 – Jun 21
	1	2	3	4	5	6	7
Course work							
Study and Review of Literature Survey							
Problem Finding							
Objective 1 Completion							
Objective 2 Completion							
Objective 3 Completion							
Documentation							

Research Methodology


Literature Review
Problem Identification
Process Planning
Data Collection and Preprocessing
Model Selection
Model Development
Model Connection Pruning
Generation of Results
Analysis of Results

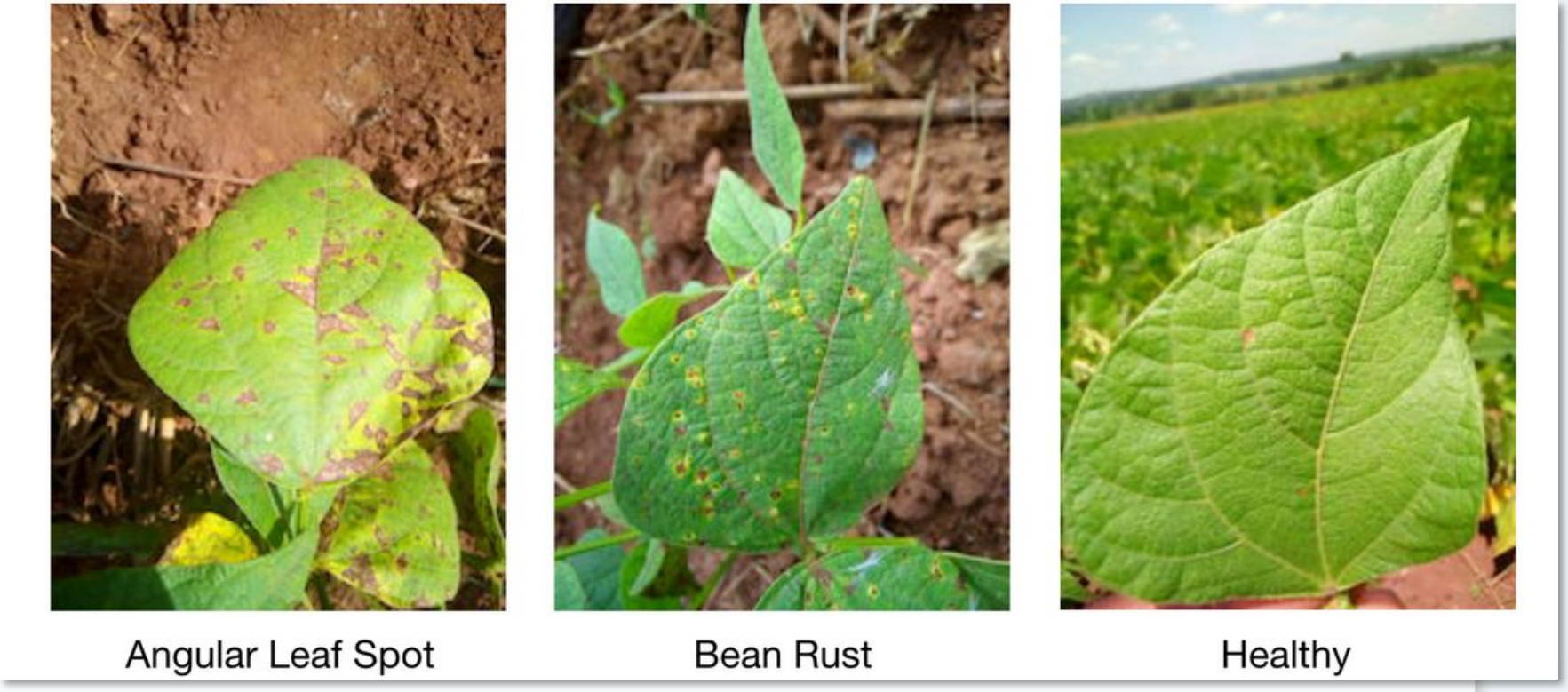
- In this research we have to study various image classification methods and find the best solution for bean leaf disease detection task.

- Python 3.7, Tensorflow 2.0 with Keras API and various libraries for classification and image processing are used.

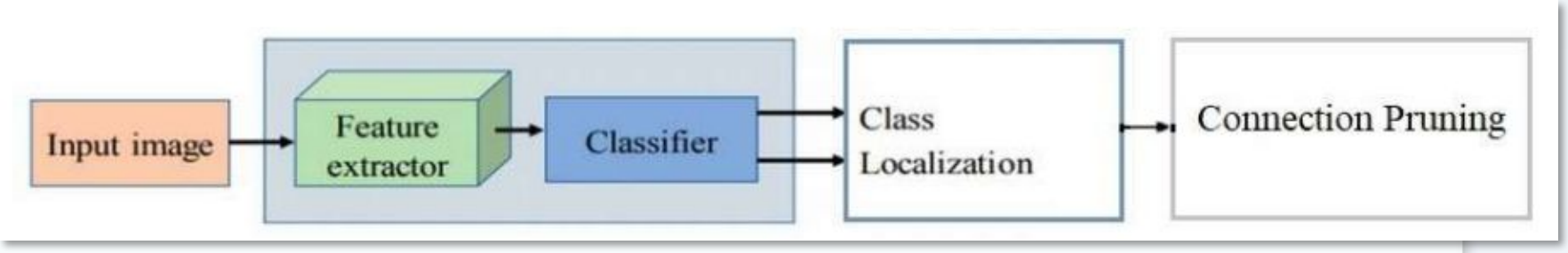
- Proposed systems is implemented on Intel i5 CPU with 2.65 GHz clock speed based machine with 2 Gb integrated graphics memory.

- Dataset -

Author => Makerere AI Lab
Title => Bean disease dataset
Month => January
Year => 2020
Url => https://www.tensorflow.org/datasets/catalog/beans

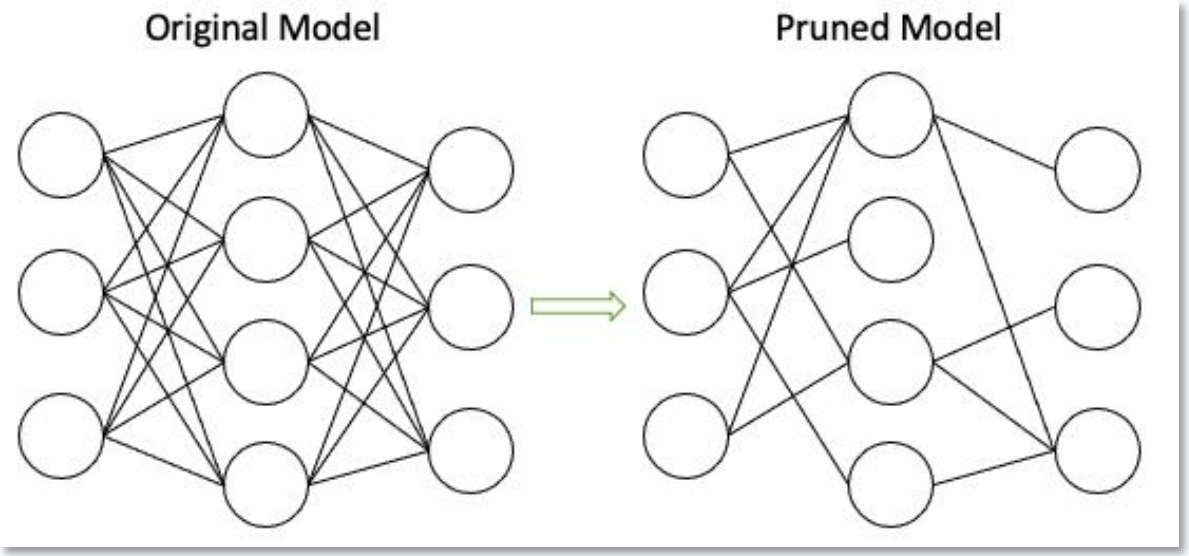


Sample images from beans dataset



Meta architecture of proposed system

Network optimization with Connection Pruning



challenging problem that needs to be addressed.

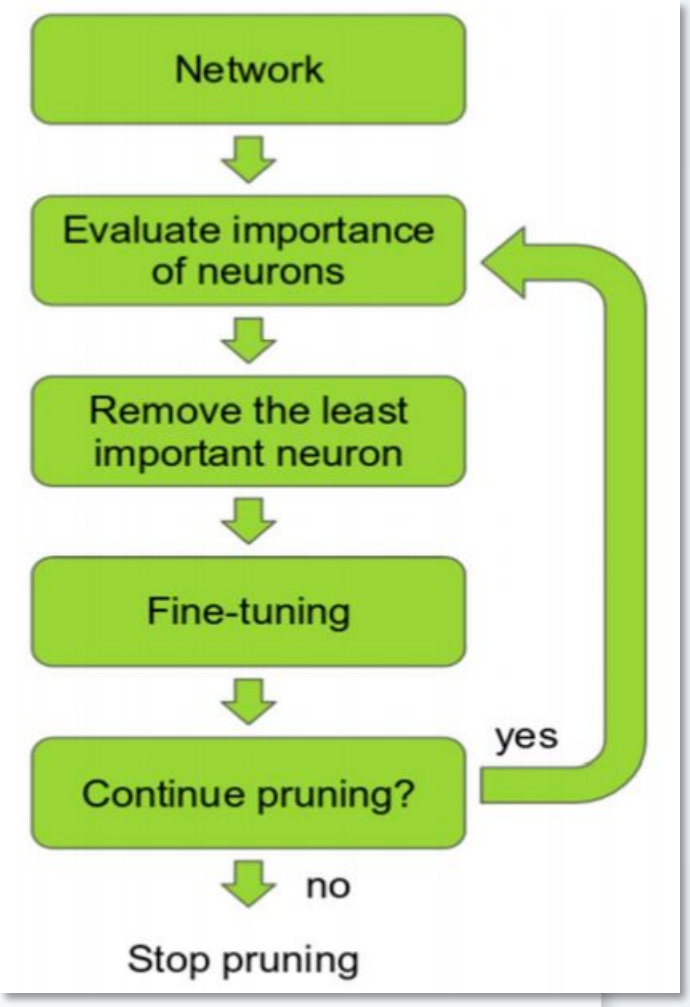
- Model pruning seeks to induce sparsity in a deep neural network's various connection matrices, thereby reducing the number of nonzero-valued parameters in the model.

Model Connection Pruning

- It is found that large-sparse models to consistently outperform small-dense models and achieve up to 10x reduction in number of non-zero parameters with minimal loss in accuracy.

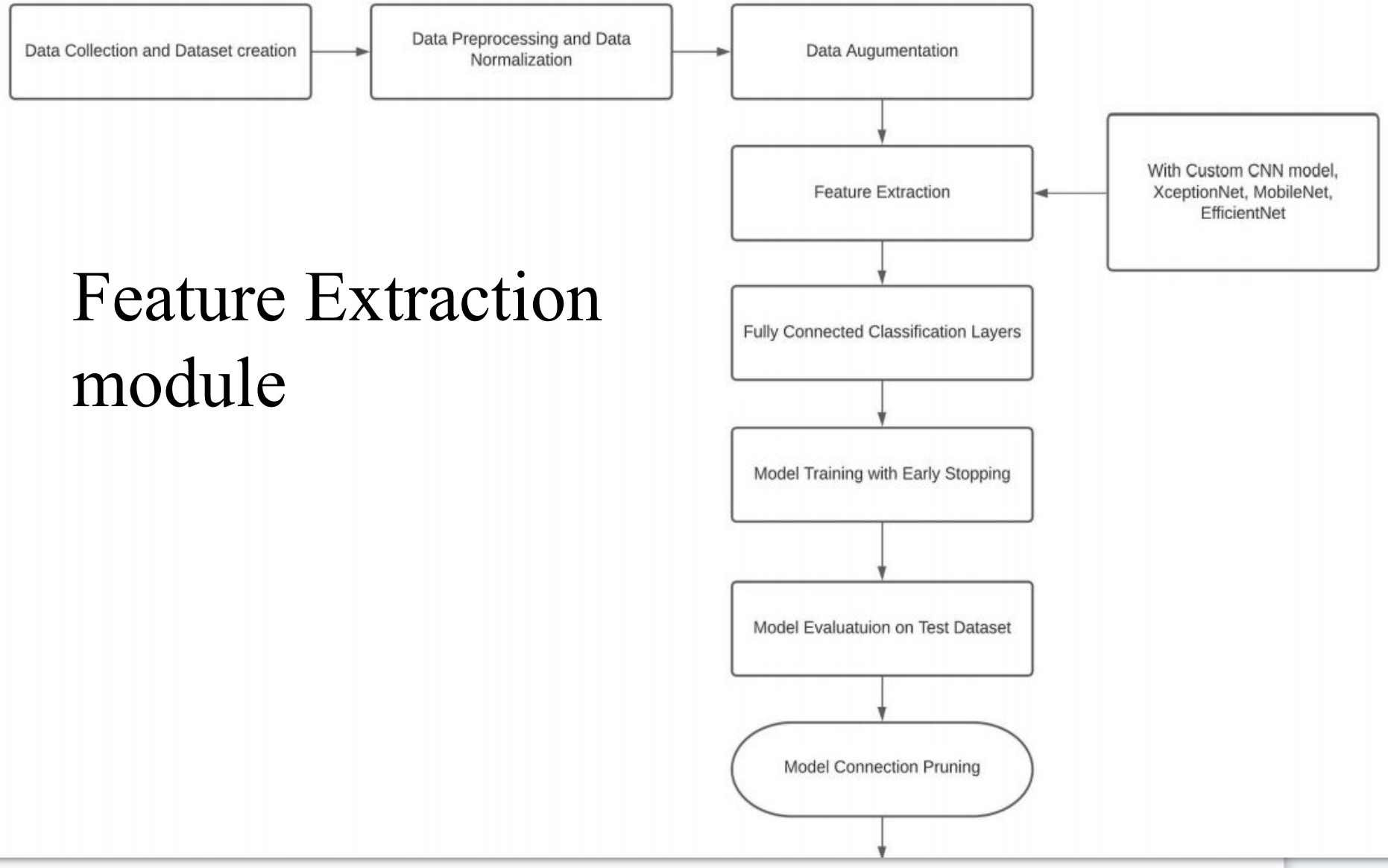
- Reduced memory consumption and computation cost.

Step involved in Connection Pruning

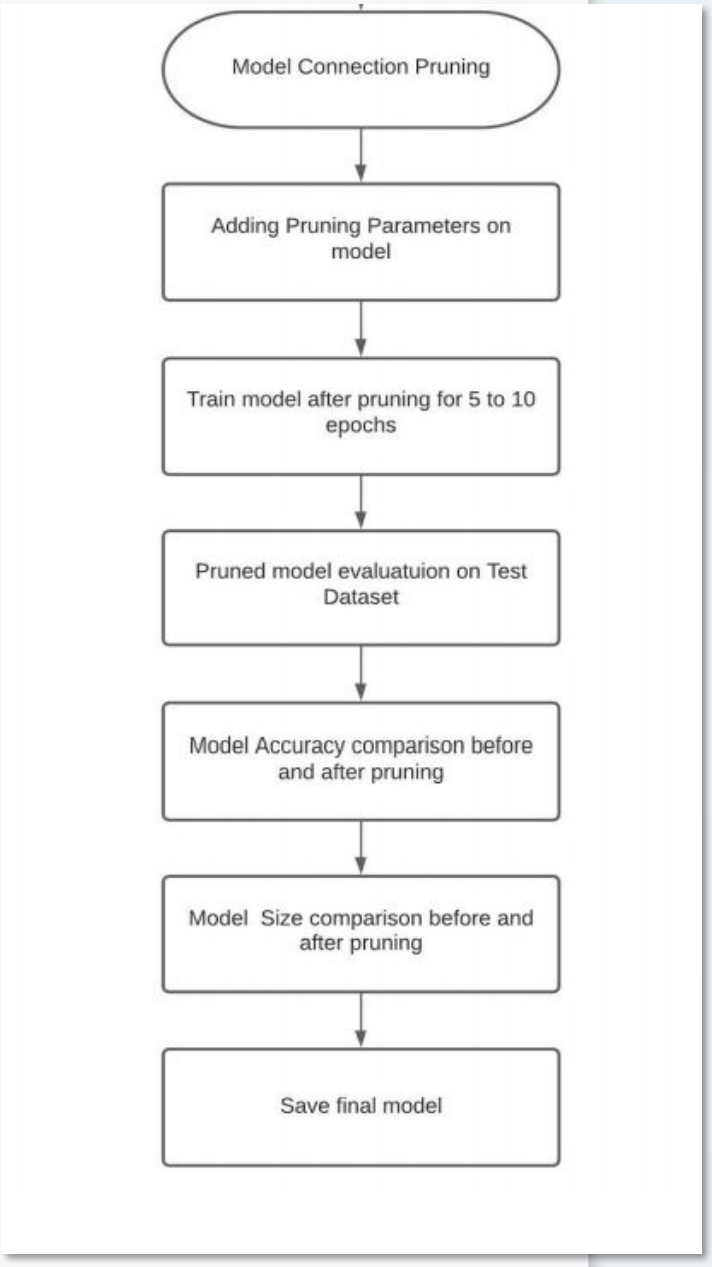


Implementation

Feature Extraction module



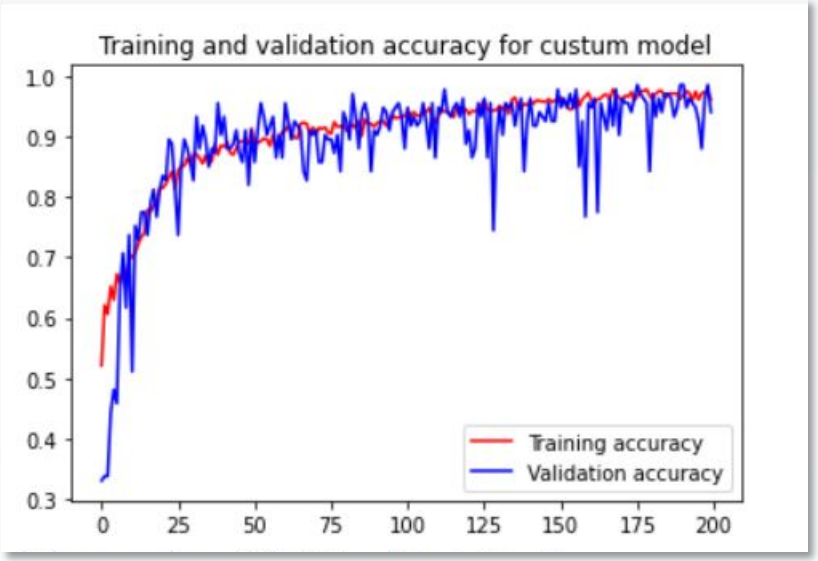
Connection Pruning module

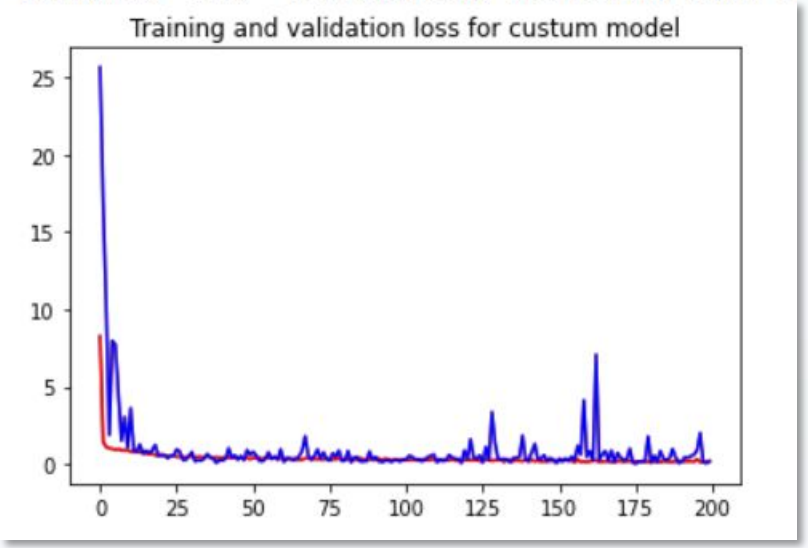


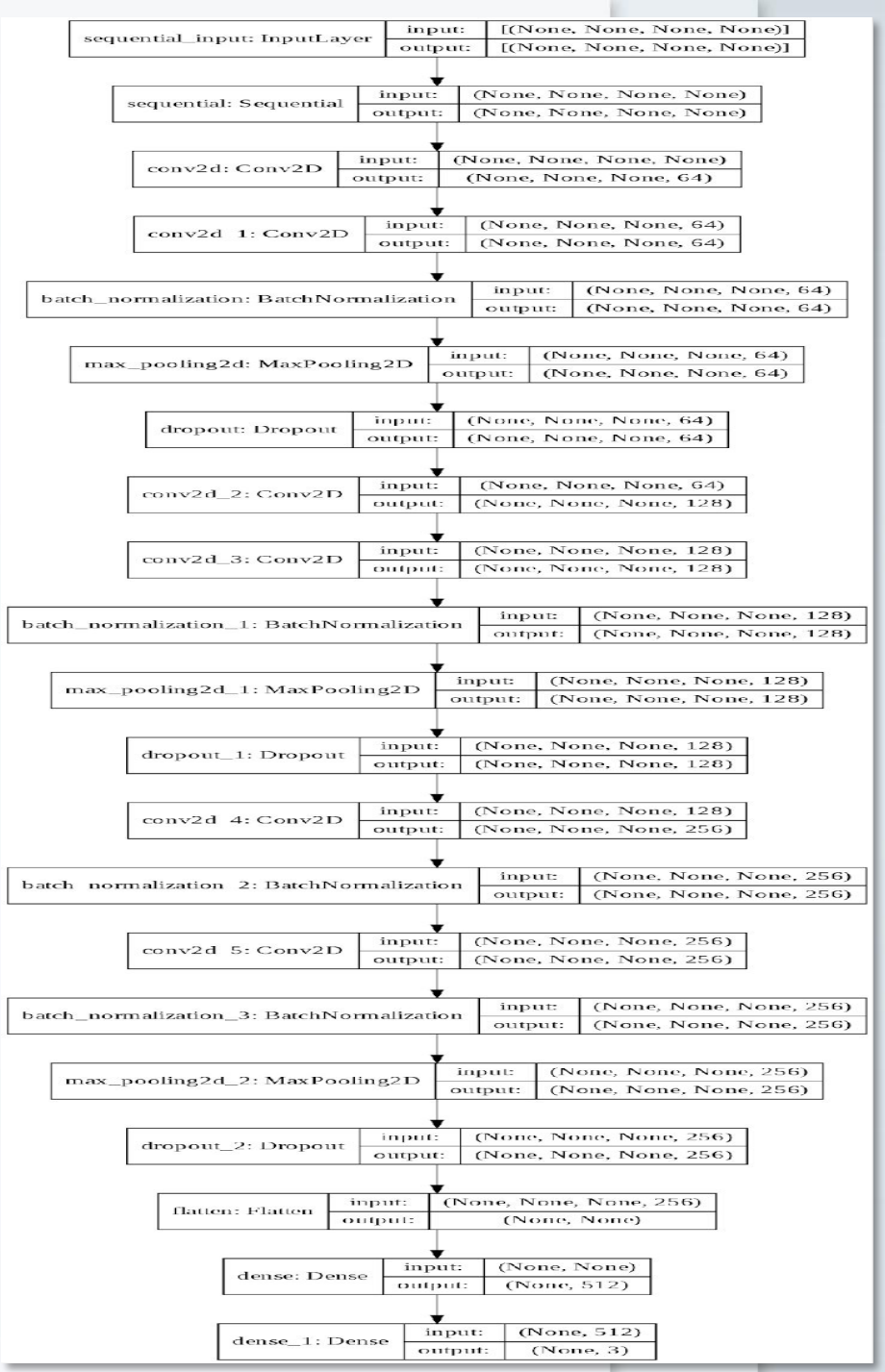
Experimental Results

AlexNet Based Custom CNN Model

- 9 Layer CNN model with batch normalization and dropouts


Training and validation accuracy for custum model
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
25
50
75
100
125
150
175
200
Training accuracy
Validation accuracy


Training and validation loss for custum model
25
20
15
10
5
0
0
25
50
75
100
125
150
175
200


sequential_input: InputLayer
input: [(None, None, None, None)]
output: [(None, None, None, None)]
sequential: Sequential
input: (None, None, None, None)
output: (None, None, None, None)
conv2d: Conv2D
input: (None, None, None, None)
output: (None, None, None, 64)
conv2d_1: Conv2D
input: (None, None, None, 64)
output: (None, None, None, 64)
batch_normalization: BatchNormalization
input: (None, None, None, 64)
output: (None, None, None, 64)
max_pooling2d: MaxPooling2D
input: (None, None, None, 64)
output: (None, None, None, 64)
dropout: Dropout
input: (None, None, None, 64)
output: (None, None, None, 64)
conv2d_2: Conv2D
input: (None, None, None, 64)
output: (None, None, None, 128)
conv2d_3: Conv2D
input: (None, None, None, 128)
output: (None, None, None, 128)
batch_normalization_1: BatchNormalization
input: (None, None, None, 128)
output: (None, None, None, 128)
max_pooling2d_1: MaxPooling2D
input: (None, None, None, 128)
output: (None, None, None, 128)
dropout_1: Dropout
input: (None, None, None, 128)
output: (None, None, None, 128)
conv2d_4: Conv2D
input: (None, None, None, 128)
output: (None, None, None, 256)
batch_normalization_2: BatchNormalization
input: (None, None, None, 256)
output: (None, None, None, 256)
conv2d_5: Conv2D
input: (None, None, None, 256)
output: (None, None, None, 256)
batch_normalization_3: BatchNormalization
input: (None, None, None, 256)
output: (None, None, None, 256)
max_pooling2d_2: MaxPooling2D
input: (None, None, None, 256)
output: (None, None, None, 256)
dropout_2: Dropout
input: (None, None, None, 256)
output: (None, None, None, 256)
flatten: Flatten
input: (None, None, None, 256)
output: (None, None)
dense: Dense
input: (None, None)
output: (None, 512)
dense_1: Dense
input: (None, 512)
output: (None, 3)

```
[96] acc = model_custom.evaluate(
         validation_generator, steps=None, callbacks=None, max_queue_size=16, workers=1,
         use_multiprocessing=False, verbose=0
     )
     print("Accuracy on validation data with custum model",acc[1]*100)
```

```
Accuracy on validation data with custum model 96.9924807548523
```

```
[97] acc = model_custom.evaluate(
         test_generator, steps=None, callbacks=None, max_queue_size=16, workers=1,
         use_multiprocessing=False, verbose=0
     )
     print("Accuracy on test data with custum model",acc[1]*100)
```

```
Accuracy on test data with custum model 92.1875
```

Test and validation accuracy of the AlexNet based custom model.

```
_, model_for_pruning_accuracy = model_for_pruning.evaluate(test_generator, verbose=2)

print('Baseline test accuracy:', baseline_model_accuracy)
print('Pruned test accuracy:', model_for_pruning_accuracy)
```

```
26/26 - 2s - loss: 0.1985 - accuracy: 0.9453
Baseline test accuracy: 0.9140625
Pruned test accuracy: 0.9453125
```

Test accuracy of the AlexNet based custom model after pruning

```
[87] print("Size of gzipped baseline custom model: %.2f mega bytes" % (get_gzipped_model_size(keras_file)/(1024*1024)))
     print("Size of gzipped pruned custom based model: %.2f mega bytes" % (get_gzipped_model_size(pruned_keras_file)/(1024*1024)))
```

```
Size of gzipped baseline custom model: 319.09 mega bytes
Size of gzipped pruned custom based model: 196.17 mega bytes
```

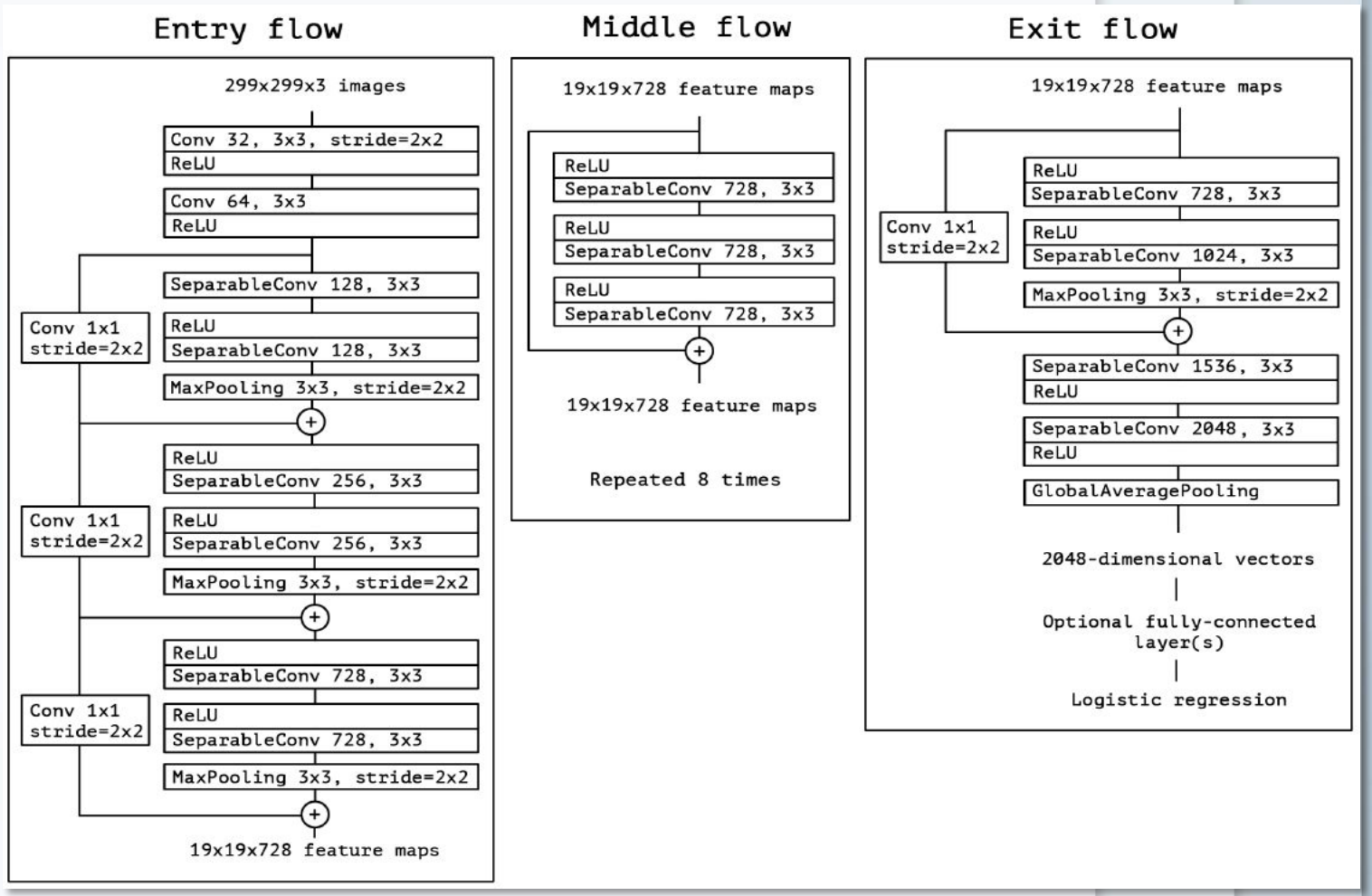
AlexNet based custom model's size before and after pruning.

Xception based feature extraction

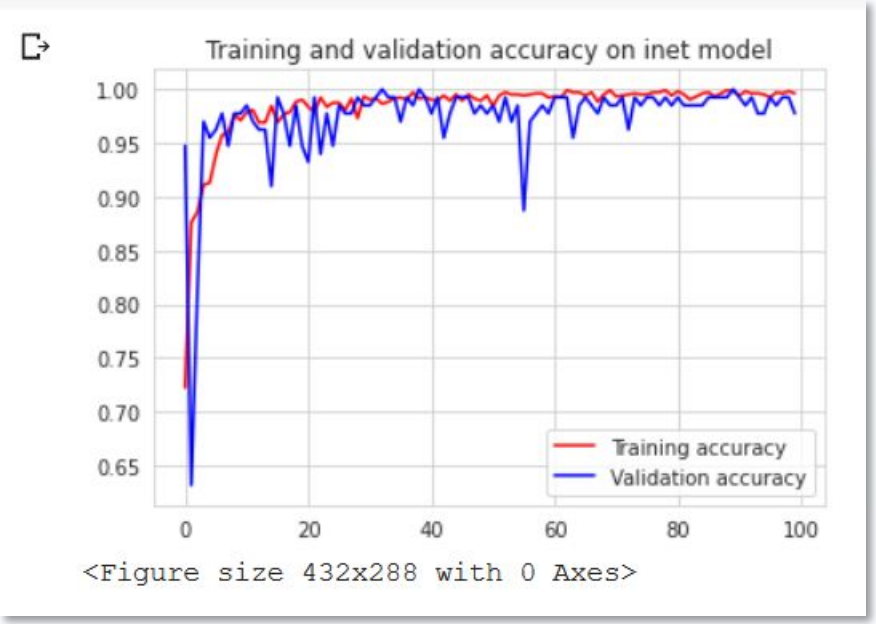
The XceptionNet architecture is based on convolution layer which are depth wise separable with connections that are residual and is 26 linearly stacked.

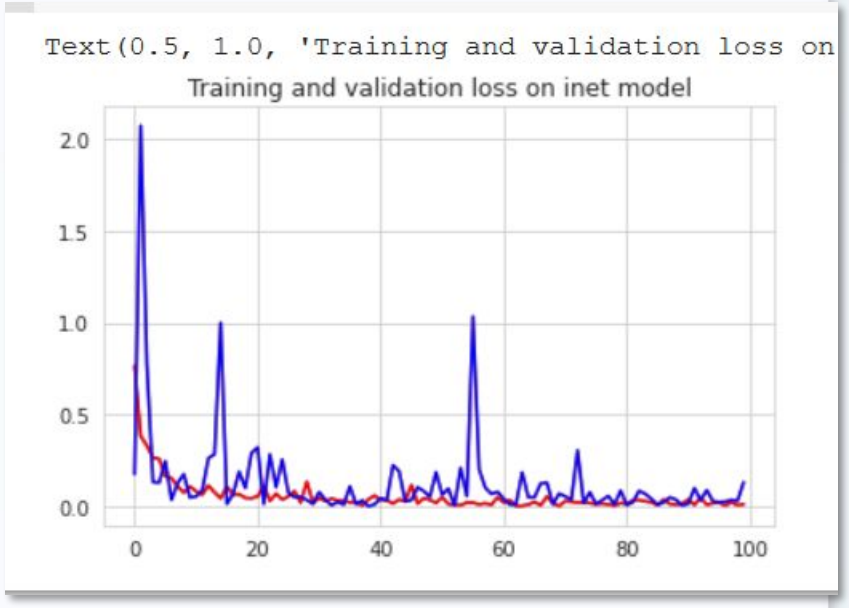
[Chollet, François. "Xception: Deep learning with depthwise separable convolutions."

In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1251-1258. (2017)]



XceptionNet based feature extraction





Train and validation accuracy-loss graphs of the XceptionNet based model.

```
[54] acc = model_i.evaluate(
         test_generator, steps=None, callbacks=None, max_queue_size=10, workers=1,
         use_multiprocessing=False, verbose=0
     )
     # acc[1]*100
```

```
[55] print("Accuracy for test set on inet ",acc[1]*100)
```

```
Accuracy for test set on inet  94.53125
```

```
acc = model_i.evaluate(
    validation_generator, steps=None, callbacks=None, max_queue_size=10, workers=1,
    use_multiprocessing=False, verbose=0
)
print("Accuracy for validation set on inet ",acc[1]*100)
```

```
Accuracy for validation set on inet  96.24060392379761
```

Test and validation accuracy of the XceptionNet based model.

```
print('Baseline test accuracy for inet:', baseline_model_accuracy)
print('Pruned test accuracy for inet:', model_for_pruning_accuracy)
```

```
Baseline test accuracy for inet: 0.9765625
Pruned test accuracy for inet: 0.953125
```

Test accuracy of the XceptionNet based model before and after pruning

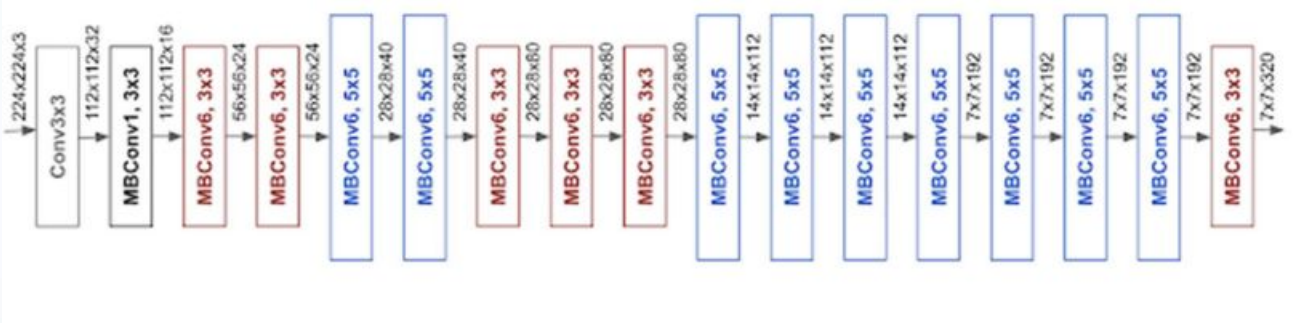
```
[44] print("Size of gzipped baseline  model: %.2f mega bytes" % (get_gzipped_model_size(keras_file)/(1024*1024)))
     print("Size of gzipped pruned inet model: %.2f mega bytes" % (get_gzipped_model_size(pruned_keras_file)/(1024*1024)))
```

```
Size of gzipped baseline  model: 80.97 mega bytes
Size of gzipped pruned inet model: 25.95 mega bytes
```

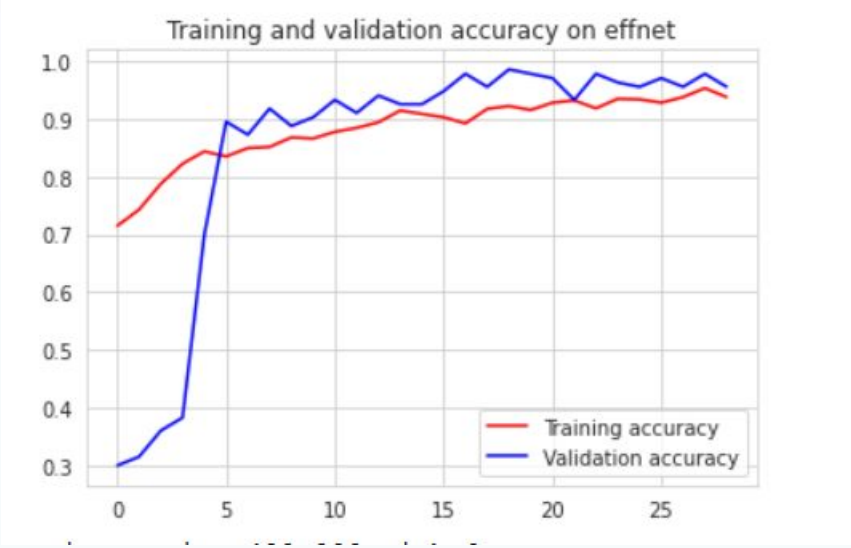
XceptionNet based model’s size before and after pruning.

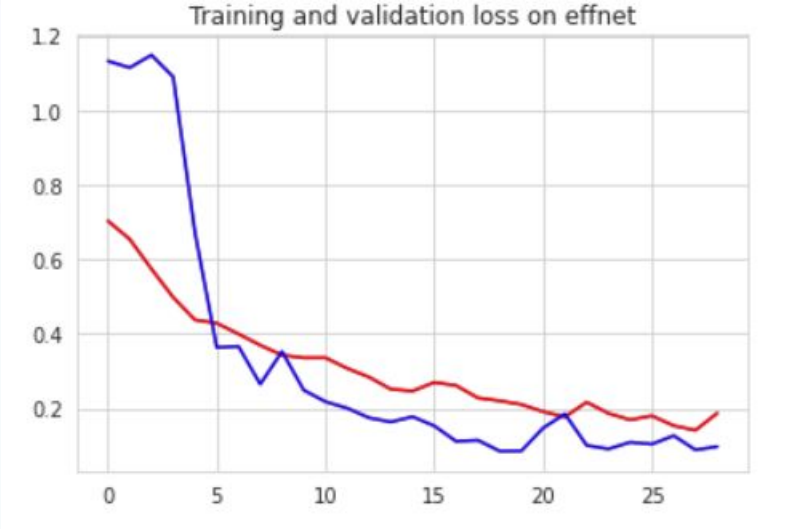
EfficientNet

- Uses a set of scaling factors to equally scale each dimension.
- Researchers have created a group of models known as EfficientNets that attains state-of-the-art accuracy with up to 10x higher efficiency, smaller and quicker.



EfficientNet based Feature extraction






```
[70] acc = model.evaluate(
         test_generator, steps=None, callbacks=None, max_queue_size=10, workers=1,
         use_multiprocessing=False, verbose=0
     )
     print("Accuracy on test with effnet-> ",acc[1]*100)
```

```
Accuracy on test with effnet->  92.1875
```

```
#Comparison of accuracy before and after model pruning

_, model_for_pruning_accuracy = model_for_pruning.evaluate(test_generator, verbose=0)

print('Baseline test accuracy:', baseline_model_accuracy)
print('Pruned test accuracy:', model_for_pruning_accuracy)
```

```
Baseline test accuracy: 0.9453125
Pruned test accuracy: 0.890625
```

Test accuracy of the EfficientNet based model before and after pruning

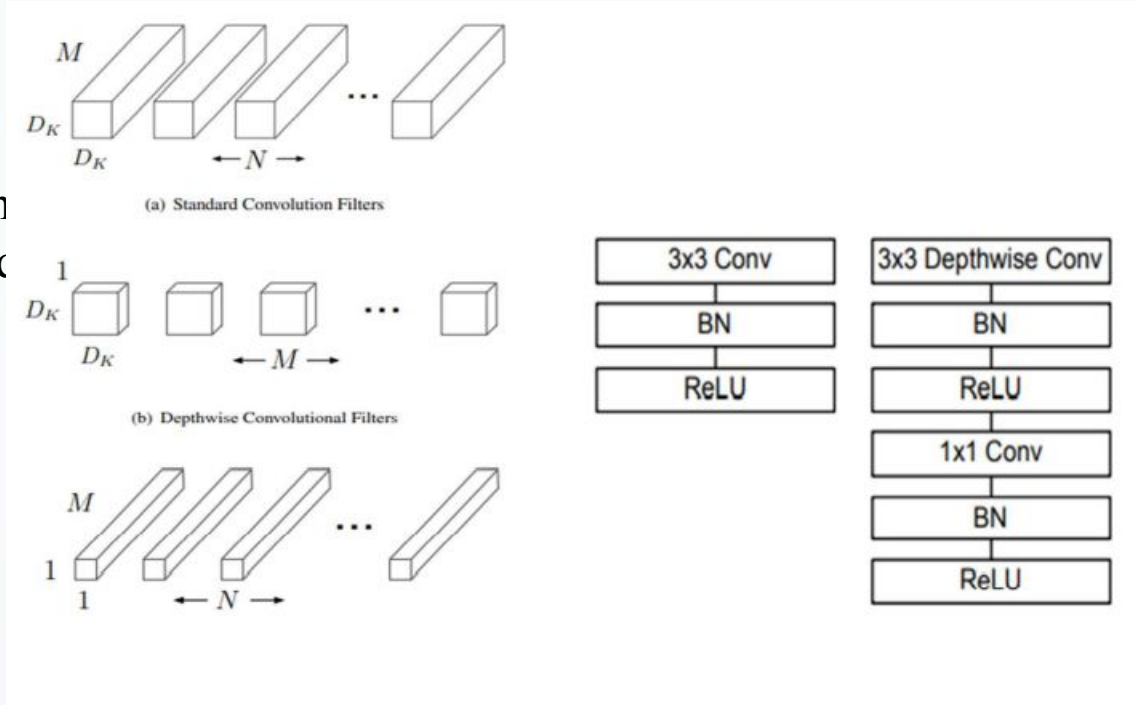
```
[86] #Comparison of model sizes before and after model pruning
     print("Size of gzipped baseline efnet model: %.2f mega bytes" % (get_gzipped_model_size(keras_file)/(1024*1024)))
     print("Size of gzipped pruned efnet model: %.2f mega bytes" % (get_gzipped_model_size(pruned_keras_file)/(1024*1024)))
```

```
Size of gzipped baseline efnet model: 16.72 mega bytes
Size of gzipped pruned efnet model: 10.56 mega bytes
```

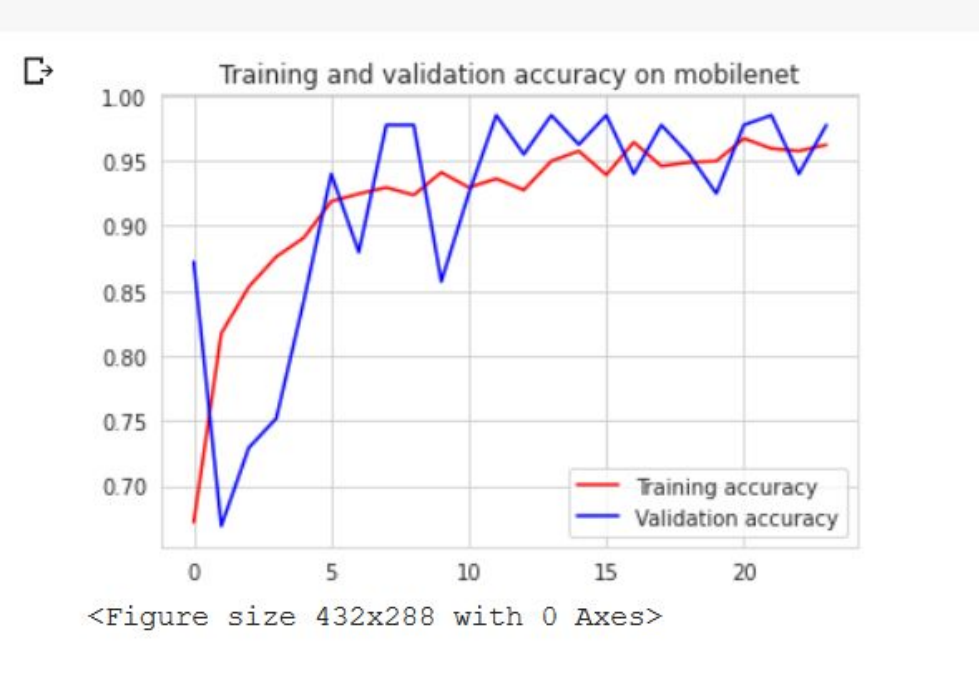
EfficientNet based model’s size before and after pruning

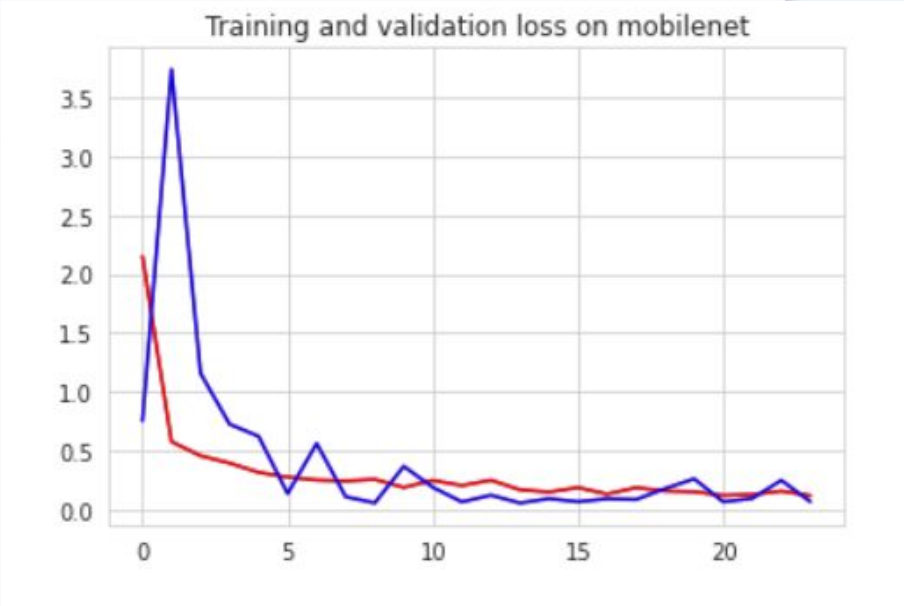
MobileNet

MobileNets are a sin thwise convolutions
and was later utilised layers.



MobileNet based Feature extraction





Train and validation accuracy-loss graphs of the mobileNet based model.

```
[33] acc = model.evaluate(
         validation_generator, steps=None, callbacks=None, max_queue_size=10, workers=1,
         use_multiprocessing=False, verbose=0
     )

     print("Accuracy on mobilenet for validation data->",acc[1]*100)
```

```
Accuracy on mobilenet for validation data-> 96.9924807548523
```

```
[34] acc = model.evaluate(
         test_generator, steps=None, callbacks=None, max_queue_size=10, workers=1,
         use_multiprocessing=False, verbose=0
     )
     acc[1]*100
     print("Accuracy on mobilenet for test data->",acc[1]*100)
```

```
Accuracy on mobilenet for test data-> 94.53125
```

Test and validation accuracy of the mobileNet based model.

```
_, model_for_pruning_accuracy = model_for_pruning.evaluate(test_generator, verbose=0)

print('Baseline test accuracy:', baseline_model_accuracy)
print('Pruned test accuracy:', model_for_pruning_accuracy)
```

```
Baseline test accuracy: 0.9453125
Pruned test accuracy: 0.96875
```

Test accuracy of the mobileNet based model before and after pruning

```
[67] print("Size of gzipped baseline mobilenet model: %.2f mega bytes" % (get_gzipped_model_size(keras_file)/(1024*1024)))
     print("Size of gzipped pruned mobilenet model: %.2f mega bytes" % (get_gzipped_model_size(pruned_keras_file)/(1024*1024)))
```

```
Size of gzipped baseline mobilenet model: 13.31 mega bytes
Size of gzipped pruned mobilenet model: 8.19 mega bytes
```

MobileNet based model’s size before and after pruning

Summary of Results with all architectures

CNN Architecture used	Accuracy on test dataset (Before Pruning)	Accuracy on test dataset (After Pruning)	Pruned model size. (In MB)	Percentage decrease in model size (After Pruning)
AlexNet based Custom CNN	92.18%	94.53%	196.17 MB	**38.52%**
XceptionNet	94.53%	95.31%	25.53 MB	**70.98%**
EfficientNet	92.18%	89.06%	10.56 MB	**63.58%**
MobileNet	94.53%	**96.87%**	8.19 MB	**48.81%**

Comparison of Training Time and Accuracy with existing methods

Title/Method used	Dataset	Accuracy	Training time / epochs
[23] M-class SVM	Grape leaf dataset	92%	-
[6] SVM	Maize leaf dataset	80%	-
[5] Modified LeNet (Without data augmentation)	Maize leaf dataset (Plant Village)	89%	1000 epochs
[4] AlexNet Model (With data augmentation)	Olive leaf dataset (Plant Village)	95%	100 epochs
[4] AlexNet Model (Without data augmentation)	Olive leaf dataset (Plant Village)	88%	100 epochs
[21] VGG Based Model	Custom dataset of apple leaves	91.7%	1000 epochs
[5] CNN + LVQ	Custom dataset of Tomato leaves	86%	300 epochs
Our proposed methodology (Feature Extraction + Connection Pruning)	**Bean Leaf dataset**	**96.87%**	**30 epochs (Early Stopping)**

Conclusion

Conclusion

- The main objective is to build a robust Machine learning model that is able to distinguish between diseases in the Bean plants.
- A two phased methodology for developing disease classification system.
- CNN based model for detecting Angular Leaf Spot and Bean Rust diseases in bean leaves with 96.87% accuracy.
- Effects of data augmentation on performance of classifier and improvement when we have less number of images for model training.
- Effect of connection pruning on different CNN architectures.

List of Publications

- Prakhar Rastogi, Arun Solanki, "*Classification Model for Detection of Diseases in Fabaceae plants using Deep Convolutional Neural Network and Connection Pruning*", International Conference on Advances in Computing, Communication and Security, NIT Kurukshetra (2021) (Communicated)

LIST OF REFERENCES

[1] Priyadharshini RA, Arivazhagan S, Arun M, Mirnalini A. Maize leaf disease classification using deep convolutional neural networks. Neural Computing and Applications. 2019 Dec;31(12):8887-95.

[2] Alehegn E (2017) Maize leaf diseases recognition and classification based on imaging and machine learning techniques. Int J Innov Res Comput Commun Eng 5(12):1–11

[3] Brahimi M, Boukhalfa K, Moussaoui A. Deep learning for tomato diseases: classification and symptoms visualization. Applied Artificial Intelligence. 2017 Apr 21;31(4):299-315.

[4] Uğuz, Sinan, and Nese Uysal. "Classification of olive leaf diseases using deep convolutional neural networks." Neural Computing and Applications (2020): 1-17.

[5] Sardogan, Melike, Adem Tuncer, and Yunus Ozen. "Plant leaf disease detection and classification based on CNN with LVQ algorithm." In 2018 3rd International Conference on Computer Science and Engineering (UBMK) , pp. 382-385. IEEE, 2018.

[6] Singh, Vijai, and Ak K. Misra. "Detection of plant leaf diseases using image segmentation and soft computing techniques." Information processing in Agriculture 4, no. 1 (2017): 41-49.

[7] Zaumeyer, William John, and H. Rex Thomas. A monographic study of bean diseases and methods for their control . No. 1488-2016-124537. 1957.

[8] Esmaeel, Anwar Ali. "A novel approach to classify and detect bean diseases based on image processing." In 2018 IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE) , pp. 297-302. IEEE, 2018.

[9] Szegedy, Christian, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. "Going deeper with convolutions." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1-9. 2015.

[10] Szegedy, Christian, Sergey Ioffe, Vincent Vanhoucke, and Alexander Alemi. "Inception-v4, inception-resnet and the impact of residual connections on learning." In Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31, no. 1. 2017.

[11] Pak, Myeongsuk, and Sanghoon Kim. "A review of deep learning in image recognition." In 2017 4th international conference on computer applications and information processing technology (CAIPT), pp. 1-3. IEEE, 2017.

[12] Simonyan, K. and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition.” CoRR abs/1409.1556 (2015): n. pag.

[13] Geoffrey Hinton Neural Networks for machine learning online course. https://www.coursera.org/learn/neural-networks/home/welcome

[14] Chollet, François. "Xception: Deep learning with depth wise separable convolutions." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1251-1258. 2017.

[15] Tan, Mingxing, and Quoc Le. "Efficientnet: Rethinking model scaling for convolutional neural networks." In International Conference on Machine Learning, pp. 6105-6114. PMLR, 2019.

[16] Park, Jongsoo, Sheng Li, Wei Wen, Ping Tak Peter Tang, Hai Li, Yiran Chen, and Pradeep Dubey. "Faster cnns with direct sparse convolutions and guided pruning." arXiv preprint arXiv:1608.01409 (2016).

[17] Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. "Dropout: a simple way to prevent neural networks from overfitting." The journal of machine learning research 15, no. 1 (2014): 1929-1958.

[18] Zhu, Michael, and Suyog Gupta. "To prune, or not to prune: exploring the efficacy of pruning for model compression." arXiv preprint arXiv:1710.01878 (2017).

- Thank you